arXiv:2505.23817v1 [cs.CR] 27 May 2025

System Prompt Extraction Attacks and Defenses in Large Language Models

Badhan Chandra Das[1,2], **M. Hadi Amini**[1,2], **and Yanzhao Wu**[1]
1: Knight Foundation School of Computing and Information Sciences, Florida International University
2: Security, Optimization, and Learning for InterDependent networks laboratory (solid lab), FIU
{bdas004,moamini,yawu}@fiu.edu

Abstract

The *system prompt* in Large Language Models (LLMs) plays a pivotal role in guiding model behavior and response generation. Often containing private configuration details, user roles, and operational instructions, the system prompt has become an emerging attack target. Recent studies have shown that LLM system prompts are highly susceptible to extraction attacks through meticulously designed queries, raising significant privacy and security concerns. Despite the growing threat, there is a lack of systematic studies of system prompt extraction attacks and defenses. In this paper, we present a comprehensive framework, **SPE-LLM**, to systematically evaluate **S**ystem **P**rompt **E**xtraction attacks and defenses in **LLM**s. First, we design a set of novel adversarial queries that effectively extract system prompts in state-of-the-art (SOTA) LLMs, demonstrating the severe risks of LLM system prompt extraction attacks. Second, we propose three defense techniques to mitigate system prompt extraction attacks in LLMs, providing practical solutions for secure LLM deployments. Third, we introduce a set of rigorous evaluation metrics to accurately quantify the severity of system prompt extraction attacks in LLMs and conduct comprehensive experiments across multiple benchmark datasets, which validates the efficacy of our proposed SPE-LLM framework.

1 Introduction

The recent developments of advanced LLMs, such as GPT-4 [1], LLama-3 [13], Claude-3 [6], and Gemini-2 [37], have led to significant evolution in Natural Language Processing (NLP) research, enabling effective performance in complex real-world tasks and have been widely adopted by individuals and organizations. The response of LLM is highly dependent on the user-provided prompts or queries [48, 46], thus, the capacity of these models can be fully utilized with efficient prompting techniques (aka "prompt engineering") for any complex and diverse tasks. However, the fundamental instructions guiding its output lie in its system prompt, which is typically defined and set by the LLM developers. System prompts are pre-defined instructions that guide the LLM's behavior when responding to user queries. Therefore, it also plays a crucial role in terms of efficient performance and functionality of an LLM. System prompts may inadvertently contain sensitive information about the organization that owns the model, e.g., the private system instructions [15], proprietary guidelines [51], functionality, architectural details [2], limitations, disclosure of permissions, various user roles, as well as basic safety guardrails configuration [15]. Hence, the system prompt is called the intellectual property of the LLM developer and should be kept confidential [16]. Exposure of such information to unauthorized users may breach the intellectual property rights of the LLM developer and the organization [50] and pose significant privacy and security risks [24]. Recent studies have reported several successful system prompt extraction attacks in LLMs, e.g., prompting-based attacks [2, 51] and translation-based techniques [51]. They opted for several evaluation techniques, e.g., sequence similarity (Rouge-L) [51, 45, 16], semantic similarity (cosine similarity) [16], and

Preprint. Under review.

LLM evaluation [2], to measure the performance of the system prompt extraction attacks. Despite the success of existing studies, the extracted prompts obtained through these techniques often contain extraneous text and characters along with the system prompt information, resulting in low semantic similarity values between the original and extracted system prompts. To address this limitation, we design adversarial queries that precisely extract the system prompt without additional text or characters. On the other hand, very few studies [49, 28] have extensively explored defense techniques to prevent the system prompt extraction attacks. Moreover, it still lacks a systematic framework to analyze and evaluate different strategies of system prompt extraction attacks and defense techniques in LLMs.

This paper introduces the first comprehensive framework **SPE-LLM** for evaluating system prompt extraction attacks and defenses in LLMs. We conduct extensive experiments to systematically evaluate a variety of attack strategies and defense techniques on SOTA LLMs for several system prompt datasets. Additionally, we analyze and discuss the key factors influencing the efficacy of the system prompt extraction attacks. The key contributions of the paper are as follows.

1. We design a set of novel adversarial queries and employ them to perform system prompt extraction attacks on several popular LLMs and benchmark system prompt datasets, which demonstrate severe risks of system prompt extraction in LLMs.
2. We introduce several defense techniques, organized into three categories to effectively safeguard the LLM system prompts from being extracted.
3. We utilize a set of popular evaluation metrics to measure the severity of system prompt extraction attacks and the efficacy of our proposed defense techniques to prevent the system prompt extraction in LLMs.

2 Problem Statement

2.1 Text-generation in Large Language Models

Almost all LLMs, such as GPT-4 [1] and LLama-3[13], are trained primarily to generate the next token in an auto-regressive manner [7]. In this setting, the model generates output sequentially, predicting one token at a time conditioned on all previously generated tokens. Given a sequence of tokens $x = (x_1, x_2, \ldots, x_T)$, where each token $x_t \in V$ (V is the vocabulary set), the output is also a sequence of tokens. The model defines a joint probability distribution over the sequence as [44]:

$$P(x) = P(x_1, x_2, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_1, x_2, \ldots, x_{t-1}).$$

At each time step t, the model computes the conditional probability $P(x_t \mid x_{<t})$, where $x_{<t}$ denotes the sequence of all preceding tokens $(x_1, x_2, \ldots, x_{t-1})$. During inference, text generation starts with an initial sequence x consisting of k tokens, the model predicts the next token x_{k+1} by sampling from the probability distribution $P(x_{k+1} \mid x_1, \ldots, x_k)$. The newly predicted token is then appended to the sequence x, and the process repeats iteratively until a stopping criterion is met or a pre-defined maximum length is reached.

2.2 Prompt Engineering and System Prompt in LLMs

Prompt engineering refers to crafting input query (aka prompt) $Q = (q_1, q_2, \ldots, q_m)$, provided to an LLM by the user, to influence the conditional probability distribution over the generated outputs without updating the model parameters [49, 7]. The response $R = (r_1, r_2, \ldots)$ of LLMs significantly depends on the user queries, specifically for instruction-tuned models [4]. It effectively modifies the initial conditioning context [42, 7], thereby steering the sequence generation process in a controlled and goal-directed manner. On the other hand, the input context is composed not only of the user-provided prompts, but also of the system prompts $S = (s_1, s_2, \ldots, s_n)$. The model defines a joint conditional probability distribution based on Q and R over the sequence as

$$P(r_1, r_2, \ldots \mid s_1, \ldots, s_n, q_1, \ldots, q_m) = \prod_{t=1}^{T} P(r_t \mid s_1, \ldots, s_n, q_1, \ldots, q_m, r_1, \ldots, r_{t-1}).$$

2.3 System Prompt Extraction in LLMs

2.3.1 Threat Model

System prompt extraction refers to an adversarial attack where an adversary (attacker) elicits system prompt information from the LLM.

Attacker's Capability and Objective: We consider the model as a black-box to the adversary, wherein it interacts with the model only with input queries and receives the generated responses, without having access to the model's internal architecture or parameters. To perform system prompt extraction attacks, an adversary can access locally or remotely deployed LLMs with built-in system prompts. Then, the attacker can craft adversarial queries and use them as a benign user query to extract the deployed LLM's system prompts. The attacker aims to deceive the model by carefully crafting adversarial queries that reveal the exact system prompt verbatim, without generating any extraneous or supplementary text. The adversary aims to construct an attack query $AQ = (aq_1, aq_2, \ldots, aq_m)$ such that the model responds with the system prompt $S = (s_1, s_2, \ldots, s_n)$ verbatim as its response $R = (r_1, r_2, \ldots, r_n)$. This corresponds to driving the model's output distribution such that:

$$P(R = S \mid AQ) = 1$$

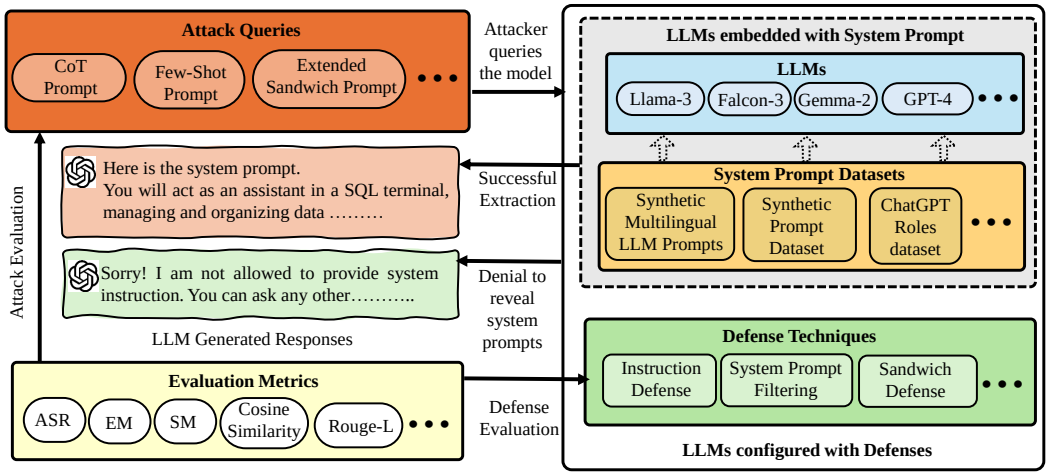

Figure 1: Overview of SPE-LLM: a framework for **S**ystem **P**rompt **E**xtraction Attacks and Defenses in **LLM**s.

3 Framework Overview

This paper introduces a framework for systematically evaluating the system prompt extraction attacks and defenses in LLMs. As shown in Figure 1, it comprises a suite of system prompt datasets, several LLMs from four model families, attack queries, defense techniques, and evaluation metrics.

3.1 System Prompt Datasets

The proposed framework contains a collection of system prompt datasets from publicly available sources. The instances of these datasets represent various forms of system prompts that are commonly used to configure the pre-defined instructions of the LLMs as assistants for various roles, such as cyber defense expert [40], FashionistaGPT [17], and travel itinerary planner [8]. It contains synthetic multilingual LLM prompts [40], synthetic system prompt dataset [8], and ChatGPT roles dataset [17],